1)Library Management Console Application :

```
class Book
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Author { get; set; }
    public bool IsAvailable { get; set; } = true;

}
class BookNotFoundException : Exception
{
    public BookNotFoundException(string message) : base(message)
    {
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.IO;
using System.Threading.Tasks;

class Program
{
    static List<Book> books = new List<Book>();
    static Dictionary<int, Book> bookDict = new Dictionary<int, Book>();
    static Queue<string> waitingUsers = new Queue<string>();
    static Stack<string> actions = new Stack<string>();

    static async Task Main()
```

```
{
    LoadBooks();

    while (true)
    {
        Console.WriteLine("\n1 Add Book");
        Console.WriteLine("2 Borrow Book");
        Console.WriteLine("3 Return Book");
        Console.WriteLine("4 Search Book");
        Console.WriteLine("5 View All Books");
        Console.WriteLine("6 Exit");

        int choice = Convert.ToInt32(Console.ReadLine());

        try
        {
            switch (choice)
            {
                case 1:
                    AddBook();
                    break;

                case 2:
                    BorrowBook();
                    break;

                case 3:
                    ReturnBook();
                    break;
```

```
            case 4:
                SearchBook();
                break;

            case 5:
                ViewBooks();
                break;

            case 6:
                await SaveBooksAsync();
                return;
        }
    }
    catch (BookNotFoundException ex)
    {
        Console.WriteLine(ex.Message);
    }
    finally
    {
        Console.WriteLine("Done");
    }
  }
}

static void AddBook()
{
  Console.Write("Enter Id: ");
  int id = Convert.ToInt32(Console.ReadLine());
```

```
    Console.Write("Enter Title: ");
    string title = Console.ReadLine();

    Console.Write("Enter Author: ");
    string author = Console.ReadLine();

    Book book = new Book { Id = id, Title = title, Author = author };

    books.Add(book);
    bookDict[id] = book;

    actions.Push("Add");
    Console.WriteLine("Book Added");
}

static void BorrowBook()
{
    Console.Write("Enter Book Id: ");
    int id = Convert.ToInt32(Console.ReadLine());

    if (!bookDict.ContainsKey(id))
        throw new BookNotFoundException("Book not found");

    Book book = bookDict[id];

    if (book.IsAvailable)
    {
        book.IsAvailable = false;
```

```
        actions.Push("Borrow");
        Console.WriteLine("Book Borrowed");
    }
    else
    {
        Console.Write("Enter your name for waiting list: ");
        string user = Console.ReadLine();
        waitingUsers.Enqueue(user);
    }
}

static void ReturnBook()
{
    Console.Write("Enter Book Id: ");
    int id = Convert.ToInt32(Console.ReadLine());

    if (!bookDict.ContainsKey(id))
        throw new BookNotFoundException("Book not found");

    Book book = bookDict[id];
    book.IsAvailable = true;

    if (waitingUsers.Count > 0)
    {
        string nextUser = waitingUsers.Dequeue();
        Console.WriteLine("Book given to " + nextUser);
    }
    else
    {
```

```
        Console.WriteLine("Book Returned");
    }
}

static void SearchBook()
{
    Console.Write("Enter Author Name: ");
    string author = Console.ReadLine();

    var result = books.Where(b => b.Author == author);

    foreach (var book in result)
    {
        Console.WriteLine(book.Title);
    }
}

static void ViewBooks()
{
    foreach (var book in books)
    {
        Console.WriteLine($"{book.Id} {book.Title} {book.Author} Available:{book.IsAvailable}");
    }
}

static void LoadBooks()
{
    if (File.Exists("books.txt"))
    {
```

```
        var lines = File.ReadAllLines("books.txt");

        foreach (var line in lines)
        {
            var data = line.Split(',');

            Book book = new Book
            {
                Id = int.Parse(data[0]),
                Title = data[1],
                Author = data[2],
                IsAvailable = bool.Parse(data[3])
            };

            books.Add(book);
            bookDict[book.Id] = book;
        }
    }
}

static async Task SaveBooksAsync()
{
    await Task.Run(() =>
    {
        List<string> lines = new List<string>();

        foreach (var book in books)
        {
            lines.Add($"{book.Id},{book.Title},{book.Author},{book.IsAvailable}");
```

```
            }

            File.WriteAllLines("books.txt", lines);
        });

        Console.WriteLine("Books saved to file");
    }
}
```

2) Student Task Tracker :-

```
class TaskItem
{
    public int TaskId { get; set; }
    public string Title { get; set; }
    public string StudentName { get; set; }
    public DateTime Deadline { get; set; }
    public bool IsCompleted { get; set; }
}
class InvalidTaskException : Exception
{
    public InvalidTaskException(string message) : base(message)
    {
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.IO;
using System.Text.Json;
using System.Threading.Tasks;
```

```
class Program
{
    static List<TaskItem> tasks = new List<TaskItem>();
    static Dictionary<int, TaskItem> taskDict = new Dictionary<int, TaskItem>();
    static HashSet<string> students = new HashSet<string>();
    static Queue<TaskItem> taskQueue = new Queue<TaskItem>();

    static void Main()
    {
        LoadTasks();

        while (true)
        {
            Console.WriteLine("\n1 Add Task");
            Console.WriteLine("2 Complete Task");
            Console.WriteLine("3 View Tasks");
            Console.WriteLine("4 Overdue Tasks");
            Console.WriteLine("5 Exit");

            int choice = Convert.ToInt32(Console.ReadLine());

            try
            {
                switch (choice)
                {
                    case 1:
                        AddTask();
                        break;
```

```
            case 2:
                Console.Write("Enter Task Id: ");
                int id = Convert.ToInt32(Console.ReadLine());
                Task.Run(() => CompleteTask(id));
                break;

            case 3:
                ViewTasks();
                break;

            case 4:
                ShowOverdueTasks();
                break;

            case 5:
                SaveTasks();
                return;
        }
    }
    catch (InvalidTaskException ex)
    {
        Console.WriteLine(ex.Message);
    }
  }
}

static void AddTask()
{
```

```
Console.Write("Task Id: ");
int id = Convert.ToInt32(Console.ReadLine());

Console.Write("Title: ");
string title = Console.ReadLine();

Console.Write("Student Name: ");
string student = Console.ReadLine();

Console.Write("Deadline (yyyy-mm-dd): ");
DateTime deadline = DateTime.Parse(Console.ReadLine());

if (deadline < DateTime.Now)
   throw new InvalidTaskException("Invalid deadline");

TaskItem task = new TaskItem
{
   TaskId = id,
   Title = title,
   StudentName = student,
   Deadline = deadline,
   IsCompleted = false
};

tasks.Add(task);
taskDict[id] = task;
students.Add(student);
taskQueue.Enqueue(task);
```

```
    Console.WriteLine("Task Added");
  }

  static void CompleteTask(int id)
  {
    if (!taskDict.ContainsKey(id))
      throw new InvalidTaskException("Task not found");

    TaskItem task = taskDict[id];
    task.IsCompleted = true;

    Console.WriteLine($"Task {id} completed by {task.StudentName}");
  }

  static void ViewTasks()
  {
    foreach (var t in tasks)
    {
      Console.WriteLine($"{t.TaskId} {t.Title} {t.StudentName} Deadline:{t.Deadline}
Completed:{t.IsCompleted}");
    }

    Console.WriteLine("Completed Tasks Count: " + tasks.Count(t => t.IsCompleted));
  }

  static void ShowOverdueTasks()
  {
    var overdue = tasks.Where(t => t.Deadline < DateTime.Now && !t.IsCompleted);
```

```
        foreach (var t in overdue)
        {
            Console.WriteLine($"{t.TaskId} {t.Title} overdue for {t.StudentName}");
        }
    }

    static void SaveTasks()
    {
        string json = JsonSerializer.Serialize(tasks);
        File.WriteAllText("tasks.json", json);
        Console.WriteLine("Tasks saved");
    }

    static void LoadTasks()
    {
        if (File.Exists("tasks.json"))
        {
            string json = File.ReadAllText("tasks.json");
            tasks = JsonSerializer.Deserialize<List<TaskItem>>(json);

            foreach (var t in tasks)
            {
                taskDict[t.TaskId] = t;
                students.Add(t.StudentName);
                taskQueue.Enqueue(t);
            }
        }
    }
}
```

3)Smart Warehouse Inventory & Order Processing System :

```
class Product
{
    public int ProductId { get; set; }
    public string ProductName { get; set; }
    public string? Category { get; set; }
    public double Price { get; set; }

    private int quantityAvailable;

    public int QuantityAvailable
    {
        get { return quantityAvailable; }
        set
        {
            if (value < 0)
                throw new Exception("Quantity cannot be negative");
            quantityAvailable = value;
        }
    }

    public Product(int id, string name, string? category, double price, int quantity)
    {
        ProductId = id;
        ProductName = name;
        Category = category;
        Price = price;
        QuantityAvailable = quantity;
    }
```

```
    public void UpdateStock(int qty)
    {
        QuantityAvailable += qty;
    }
}
class Order
{
    public int OrderId { get; set; }
    public string CustomerName { get; set; }
    public DateTime OrderDate { get; set; }

    public List<(int ProductId, int Quantity)> Products { get; set; }

    public Order()
    {
        Products = new List<(int, int)>();
        OrderDate = DateTime.Now;
    }
}
class OutOfStockException : Exception
{
    public OutOfStockException(string message) : base(message) { }
}
class InventoryRepository
{
    public Dictionary<int, Product> inventory = new Dictionary<int, Product>();

    public void AddProduct(Product product)
```

```
    {
        inventory[product.ProductId] = product;
    }

    public Product GetProduct(int id)
    {
        if (!inventory.ContainsKey(id))
            throw new Exception("Product not found");

        return inventory[id];
    }

    public void ViewInventory()
    {
        foreach (var p in inventory.Values)
        {
            Console.WriteLine($"{p.ProductId} {p.ProductName} {p.Price} Qty:{p.QuantityAvailable}");
        }
    }
}
class OrderService
{
    Queue<Order> orderQueue = new Queue<Order>();
    Stack<Order> shippedOrders = new Stack<Order>();
    InventoryRepository repo;

    public OrderService(InventoryRepository r)
    {
        repo = r;
```

```
}

public void PlaceOrder(Order order)
{
    orderQueue.Enqueue(order);
}

public async Task ProcessOrdersAsync()
{
    while (orderQueue.Count > 0)
    {
        Order order = orderQueue.Dequeue();

        await Task.Run(() =>
        {
            foreach (var item in order.Products)
            {
                var product = repo.GetProduct(item.ProductId);

                if (product.QuantityAvailable < item.Quantity)
                    throw new OutOfStockException("Stock not available");

                product.QuantityAvailable -= item.Quantity;
            }

            shippedOrders.Push(order);
            Console.WriteLine($"Order {order.OrderId} processed");
        });
    }
```

```
    }
}
static void Reports(Dictionary<int, Product> inventory)
{
    var products = inventory.Values;

    var lowStock = products.Where(p => p.QuantityAvailable < 10);

    var expensive = products.OrderByDescending(p => p.Price).FirstOrDefault();

    var sorted = products.OrderBy(p => p.Price);

    var categoryCount = products.GroupBy(p => p.Category)
                        .Select(g => new { Category = g.Key, Count = g.Count() });

    Console.WriteLine("Low Stock Products:");
    foreach (var p in lowStock)
        Console.WriteLine(p.ProductName);

    Console.WriteLine($"Most Expensive Product: {expensive?.ProductName}");
}
static void SaveInventory(Dictionary<int, Product> inventory)
{
    using (StreamWriter writer = new StreamWriter("inventory.txt"))
    {
        foreach (var p in inventory.Values)
        {
            writer.WriteLine($"{p.ProductId},{p.ProductName},{p.Category},{p.Price},{p.QuantityAvailable}");
        }
```

```
    }
}

static void LoadInventory(Dictionary<int, Product> inventory)
{
    if (!File.Exists("inventory.txt"))
        return;

    using (StreamReader reader = new StreamReader("inventory.txt"))
    {
        string line;

        while ((line = reader.ReadLine()) != null)
        {
            var data = line.Split(',');

            Product p = new Product(
                int.Parse(data[0]),
                data[1],
                data[2],
                double.Parse(data[3]),
                int.Parse(data[4])
            );

            inventory[p.ProductId] = p;
        }
    }
}
static async Task Main()
```

```
{
    InventoryRepository repo = new InventoryRepository();
    OrderService service = new OrderService(repo);

    while (true)
    {
        Console.WriteLine("1 Add Product");
        Console.WriteLine("2 View Inventory");
        Console.WriteLine("3 Place Order");
        Console.WriteLine("4 Process Orders");
        Console.WriteLine("5 Exit");

        int choice = int.Parse(Console.ReadLine());

        switch (choice)
        {
            case 1:
                repo.AddProduct(new Product(1, "Laptop", "Electronics", 50000, 20));
                break;

            case 2:
                repo.ViewInventory();
                break;

            case 3:
                Order order = new Order { OrderId = 1, CustomerName = "John" };
                order.Products.Add((1, 2));
                service.PlaceOrder(order);
                break;
```

```
            case 4:
                await service.ProcessOrdersAsync();
                break;

            case 5:
                return;
        }
    }
}
```

1st Code - Parallel API Data Processor (Async + Array Methods) :

```
// Fake APIs
function getUsers() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve([
        { id: 1, name: "Rahul" },
        { id: 2, name: "Anita" }
      ]);
    }, 1000);
  });
}

function getOrders() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve([
        { userId: 1, amount: 500 },
        { userId: 1, amount: 800 },
```

```
      { userId: 2, amount: 300 }
    ]);
  }, 1200);
 });
}

function getPayments() {
 return new Promise(resolve => {
  setTimeout(() => {
    resolve([
     { paymentId: 1, status: "success" }
    ]);
  }, 1500);
 });
}

// Main function
async function processData() {

 // Fetch all APIs in parallel
 const [users, orders, payments] = await Promise.all([
  getUsers(),
  getOrders(),
  getPayments()
 ]);

 // Calculate total order amount per user
 const result = users.map(user => {
  const total = orders
```

```
    .filter(order => order.userId === user.id)
    .reduce((sum, order) => sum + order.amount, 0);

  return { name: user.name, total };
 });

 // Print result
 result.forEach(r => {
  console.log(`${r.name} → ${r.total}`);
 });
}

// Run
processData();
```

2nd CODE – Smart Array Grouping System :

```
const transactions = [
 {category:"Food", amount:200},
 {category:"Travel", amount:500},
 {category:"Food", amount:300},
 {category:"Shopping", amount:800},
 {category:"Travel", amount:200}
];

// Step 1: Group and sum amounts using reduce
const grouped = transactions.reduce((acc, item) => {
 acc[item.category] = (acc[item.category] || 0) + item.amount;
 return acc;
}, {});
```

```
// Step 2: Convert object to array
const result = Object.entries(grouped);

// Step 3: Sort by highest spending
result.sort((a,b) => b[1] - a[1]);

// Step 4: Print output
result.forEach(([category,total]) => {
  console.log(`${category} → ${total}`);
});
```

3rd CODE - Async Task Queue (Run 3 Tasks at a Time) :

```
const tasks = Array.from({ length: 10 }, (_, i) => i + 1);
function runTask(id) {
  return new Promise(resolve => {
    console.log("Starting Task", id);

    const time = Math.floor(Math.random() * 2000) + 1000;

    setTimeout(() => {
      console.log("Completed Task", id);
      resolve();
    }, time);
  });
}

async function processTasks(taskList, limit) {

  const queue = [...taskList];
```

```
  const running = [];

  while (queue.length > 0 || running.length > 0) {

    while (running.length < limit && queue.length > 0) {

      const taskId = queue.shift();

      const taskPromise = runTask(taskId).then(() => {
        running.splice(running.indexOf(taskPromise), 1);
      });

      running.push(taskPromise);
    }

    await Promise.race(running);
  }
}
processTasks(tasks, 3);
```

4th CODE - Dynamic Table Filter :

```
<!DOCTYPE html>
<html>
<head>
<title>Employee Table</title>
</head>

<body>

<h2>Employee List</h2>
```

```
<label>Department:</label>
<select id="deptFilter">
<option value="All">All</option>
<option value="IT">IT</option>
<option value="HR">HR</option>
<option value="Finance">Finance</option>
</select>

<table border="1" id="empTable">
<thead>
<tr>
<th>Name</th>
<th>Department</th>
<th>Salary</th>
</tr>
</thead>
<tbody></tbody>
</table>

<h3 id="avgSalary"></h3>

<script>

const employees = [
 {name:"Rahul", department:"IT", salary:60000},
 {name:"Anita", department:"HR", salary:50000},
 {name:"Karan", department:"IT", salary:70000},
 {name:"Priya", department:"Finance", salary:65000}
```

```
];

const tableBody = document.querySelector("#empTable tbody");
const filter = document.getElementById("deptFilter");
const avgDisplay = document.getElementById("avgSalary");


function renderTable(data){

 tableBody.innerHTML = "";

 data.forEach(emp => {
  const row = document.createElement("tr");

  row.innerHTML = `
   <td>${emp.name}</td>
   <td>${emp.department}</td>
   <td>${emp.salary}</td>
  `;

  tableBody.appendChild(row);
 });

 const avg = data
   .map(e => e.salary)
   .reduce((sum,val)=>sum+val,0) / data.length || 0;

 avgDisplay.textContent = "Average Salary: " + avg;
}
```

```
}
filter.addEventListener("change", function(){

 const dept = this.value;

 const filtered = dept === "All"
  ? employees
  : employees.filter(e => e.department === dept);

 renderTable(filtered);

});

renderTable(employees);

</script>

</body>
</html>
```

5th CODE - 5Real-Time Search Suggestion :

```
<!DOCTYPE html>
<html>
<body>

<input type="text" id="search" placeholder="Search product">
<ul id="suggestions"></ul>

<script>
const products=[
```

```
"laptop",
"lamp",
"laser printer",
"keyboard",
"mouse",
"monitor",
"mobile"
];

const box=document.getElementById("search");
const list=document.getElementById("suggestions");

box.addEventListener("input",()=>{
const text=box.value.toLowerCase();
list.innerHTML="";

if(!text) return;

const matches=products.filter(p=>p.toLowerCase().startsWith(text));

matches.forEach(p=>{
const item=document.createElement("li");
item.innerHTML=p.replace(new RegExp(text,"i"),m=>`<b>${m}</b>`);
list.appendChild(item);
});
});
</script>

</body>
```

```
</html>
```

6th CODE - Advanced Array Challenge :

```
const arr = [
 [1,2,3],
 [4,5],
 [6,7,8,9]
];

const result = [...new Set(arr.flat())]
  .filter(n => n % 2 === 0)
  .sort((a,b) => b - a);

console.log(result);
```

1st CODE - 1Persistent Multi-Step Form (LocalStorage + Navigation + Validation) :

HTML :

```
<!DOCTYPE html>
<html>
<head>
<title>Multi Step Form</title>
<style>
.step{display:none;}
.active{display:block;}
</style>
</head>

<body>

<h2>Profile Form</h2>
```

```
<!-- STEP 1 -->
<div class="step" id="step1">
<h3>Step 1 – Personal Info</h3>

<label>Name:</label>
<input type="text" id="name"><br><br>

<label>Email:</label>
<input type="email" id="email"><br><br>

<label>Age:</label>
<input type="number" id="age"><br><br>

<button onclick="nextStep(1)">Next</button>
</div>


<!-- STEP 2 -->
<div class="step" id="step2">
<h3>Step 2 – Skills</h3>

<input list="skillsList" id="skillInput">
<datalist id="skillsList">
<option value="JavaScript">
<option value="Python">
<option value="Java">
<option value="Go">
</datalist>
```

```
<button onclick="addSkill()">Add Skill</button>

<ul id="skills"></ul>

<br>
<button onclick="prevStep(2)">Previous</button>
<button onclick="nextStep(2)">Next</button>
</div>

<!-- STEP 3 -->
<div class="step" id="step3">
<h3>Step 3 – Profile Strength</h3>

<progress id="profileProgress" value="0" max="100"></progress>
<p id="percent"></p>

<button onclick="prevStep(3)">Previous</button>
</div>

<script src="script.js"></script>

</body>
</html>
```

JAVA SCRIPT :

```
let currentStep = 1
let skills = JSON.parse(localStorage.getItem("skills")) || []
```

```
function showStep(step)
{
   document.querySelectorAll(".step").forEach(s=>s.classList.remove("active"))
   document.getElementById("step"+step).classList.add("active")
   currentStep = step
   localStorage.setItem("step",step)
}

function nextStep(step)
{
   if(step===1)
   {
      let name=document.getElementById("name").value
      let email=document.getElementById("email").value
      let age=document.getElementById("age").value

      if(!name || !email || !age)
      {
         alert("Fill all fields")
         return
      }
   }

   if(step===2 && skills.length===0)
   {
      alert("Add at least one skill")
      return
   }
```

```
    showStep(step+1)
    updateProgress()
}

function prevStep(step)
{
    showStep(step-1)
}

function addSkill()
{
    let skill=document.getElementById("skillInput").value

    if(skill)
    {
        skills.push(skill)
        localStorage.setItem("skills",JSON.stringify(skills))
        renderSkills()
        document.getElementById("skillInput").value=""
    }
}

function renderSkills()
{
    let list=document.getElementById("skills")
    list.innerHTML=""

    skills.forEach(s=>{
        let li=document.createElement("li")
```

```
        li.textContent=s
        list.appendChild(li)
    })
}

function saveData()
{
    localStorage.setItem("name",document.getElementById("name").value)
    localStorage.setItem("email",document.getElementById("email").value)
    localStorage.setItem("age",document.getElementById("age").value)
}

function loadData()
{
    document.getElementById("name").value=localStorage.getItem("name") || ""
    document.getElementById("email").value=localStorage.getItem("email") || ""
    document.getElementById("age").value=localStorage.getItem("age") || ""

    renderSkills()

    let step=localStorage.getItem("step") || 1
    showStep(step)

    updateProgress()
}

function updateProgress()
{
    let progress=0
```

```
    if(document.getElementById("name").value) progress+=25
    if(document.getElementById("email").value) progress+=25
    if(document.getElementById("age").value) progress+=25
    if(skills.length>0) progress+=25

    document.getElementById("profileProgress").value=progress
    document.getElementById("percent").innerText=progress+"% Complete"
}

document.querySelectorAll("input").forEach(i=>{
    i.addEventListener("input",()=>{
        saveData()
        updateProgress()
    })
})

window.onload=loadData
```

2)Smart Responsive Product Grid (CSS Grid + Dynamic Filtering) :

HTML :

```
<!DOCTYPE html>
<html>
<head>
<title>Product Catalog</title>
<link rel="stylesheet" href="style.css">
</head>

<body>
```

```
<h2>Product Catalog</h2>

<select id="filter">
<option value="all">All</option>
<option value="electronics">Electronics</option>
<option value="fashion">Fashion</option>
</select>

<div class="grid" id="productGrid">

<div class="product electronics">Laptop</div>
<div class="product fashion">Shoes</div>
<div class="product fashion">Watch</div>
<div class="product electronics">Phone</div>
<div class="product electronics">Camera</div>

</div>

<script src="script.js"></script>

</body>
</html>
```

CSS :

```
body{
font-family: Arial;
padding:20px;
}

.grid{
```

```
display:grid;
grid-template-columns:repeat(4,1fr);
gap:20px;
margin-top:20px;
}

.product{
padding:20px;
background:#f4f4f4;
text-align:center;
border-radius:6px;
transition:opacity 0.4s ease, transform 0.4s ease;
}

/* hidden state */
.hide{
opacity:0;
transform:scale(0.8);
pointer-events:none;
}

/* Tablet */
@media(max-width:900px){
.grid{
grid-template-columns:repeat(2,1fr);
}
}

/* Mobile */
```

```
@media(max-width:500px){
.grid{
grid-template-columns:1fr;
}
}
```

JAVA SCRIPT :

```
const filter=document.getElementById("filter")
const products=document.querySelectorAll(".product")

filter.addEventListener("change",function(){

let category=this.value

products.forEach(product=>{

if(category==="all"){
product.classList.remove("hide")
}
else if(product.classList.contains(category)){
product.classList.remove("hide")
}
else{
product.classList.add("hide")
}

})

})
```

3) Location-Based Weather Preference System :

HMTL :

```
<!DOCTYPE html>
<html>
<head>
<title>Weather Preference</title>
</head>

<body>

<h2>Location Based Weather</h2>

<button id="locationBtn">Get My Location</button>

<p id="locationResult"></p>

<h3>Select Temperature Unit</h3>

<label>
<input type="radio" name="unit" value="celsius"> Celsius
</label>

<label>
<input type="radio" name="unit" value="fahrenheit"> Fahrenheit
</label>

<script src="script.js"></script>

</body>
</html>
```

JAVA SCRIPT :

```
const button = document.getElementById("locationBtn")
const result = document.getElementById("locationResult")
const units = document.querySelectorAll("input[name='unit']")

// Get location
button.addEventListener("click", () => {

if (!navigator.geolocation) {
result.innerText = "Geolocation not supported"
return
}

navigator.geolocation.getCurrentPosition(showPosition, showError)

})

function showPosition(position) {

let lat = position.coords.latitude
let lon = position.coords.longitude

result.innerHTML = `
Latitude: ${lat} <br>
Longitude: ${lon}
`
}

// Error Handling
```

```
function showError(error) {
switch (error.code) {
case error.PERMISSION_DENIED:
result.innerText = "User denied the location request."
break
case error.POSITION_UNAVAILABLE:
result.innerText = "Location information unavailable."
break
case error.TIMEOUT:
result.innerText = "The request to get location timed out."
break

default:
result.innerText = "Unknown error occurred."
}

}

// Save temperature preference
units.forEach(unit => {
unit.addEventListener("change", () => {
sessionStorage.setItem("temperatureUnit", unit.value)

})
})

// Restore preference on reload
window.onload = () => {
```

```
let savedUnit = sessionStorage.getItem("temperatureUnit")

if(savedUnit){

document.querySelector(`input[value="${savedUnit}"]`).checked = true

}

}
```